CS 419: Secure Web Application

SECURITY DESIGN DOCUMENT

Secure Document Sharing System

Spring 2026 | CS 419

A. Architecture Overview

A.1 System Architecture

The application is a single-process Flask web server. All security decisions are made inside the application process before any filesystem access occurs. The trust boundary is clean: the browser may only reach the filesystem through Flask, and Flask enforces authentication, authorization, and input validation on every route.

Browser (Client)

↕ *HTTPS / TLS (port 5000)*

Flask Application (app.py)

`Routes & Jinja2 Templates` - URL dispatch, HTML rendering
`require_auth / require_role decorators` - Authentication & RBAC gate
`get_current_session()` - Session validation & timeout
`EncryptedStorage (Fernet)` - Encrypt/decrypt all documents
`SecurityLogger / AccessLogger` - Structured JSON event logging
`is_rate_limited(ip)` - In-memory IP rate limiting
`set_security_headers()` - CSP, HSTS, X-Frame-Options, etc.

↕ *Filesystem I/O*

`data/users.json` - bcrypt password hashes, roles, lockout state
`data/sessions.json` - Server-side session tokens & timestamps
`data/<uuid>.enc` - Fernet-encrypted document blobs (base64)
`logs/security.log` - Auth, access denied, lockout events
`logs/access.log` - Normal document operations
`secret.key` - Fernet symmetric key (local file)

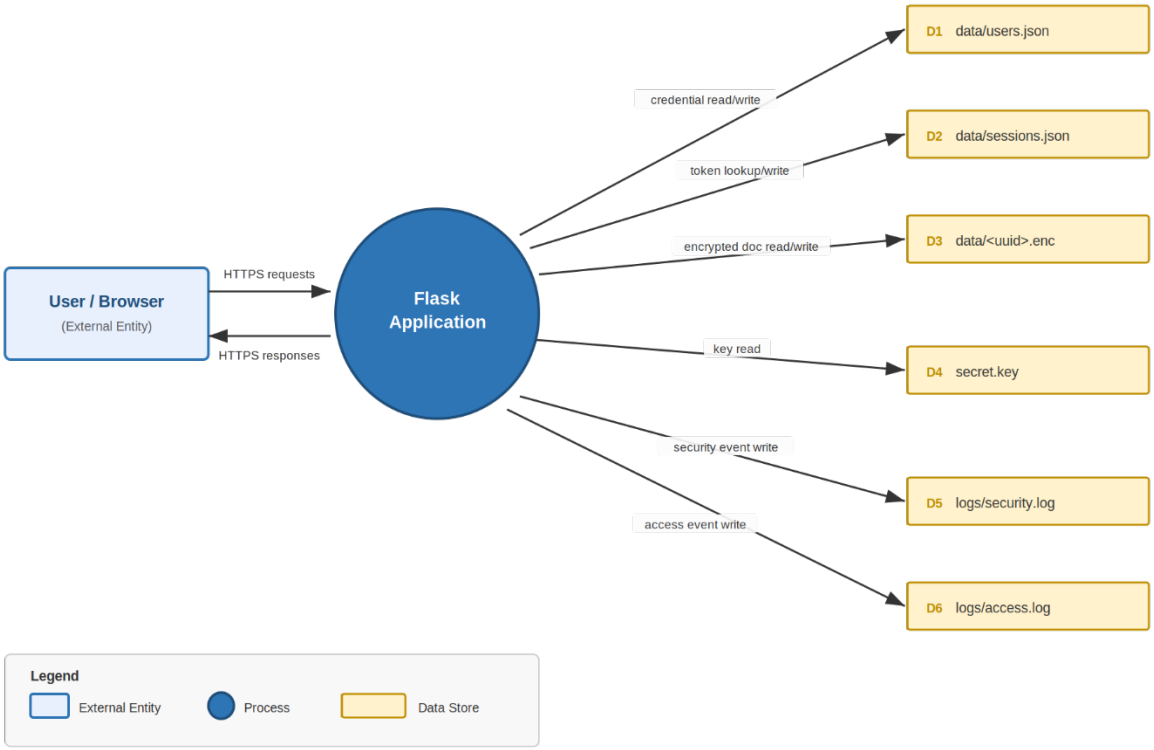
A.2 Data Flow Diagrams

The system's data flows are presented as standard DFD diagrams, followed by step-by-step flow details.

A.2.1 Context-Level DFD

The context-level DFD shows the system boundary, external entities, and all data stores.

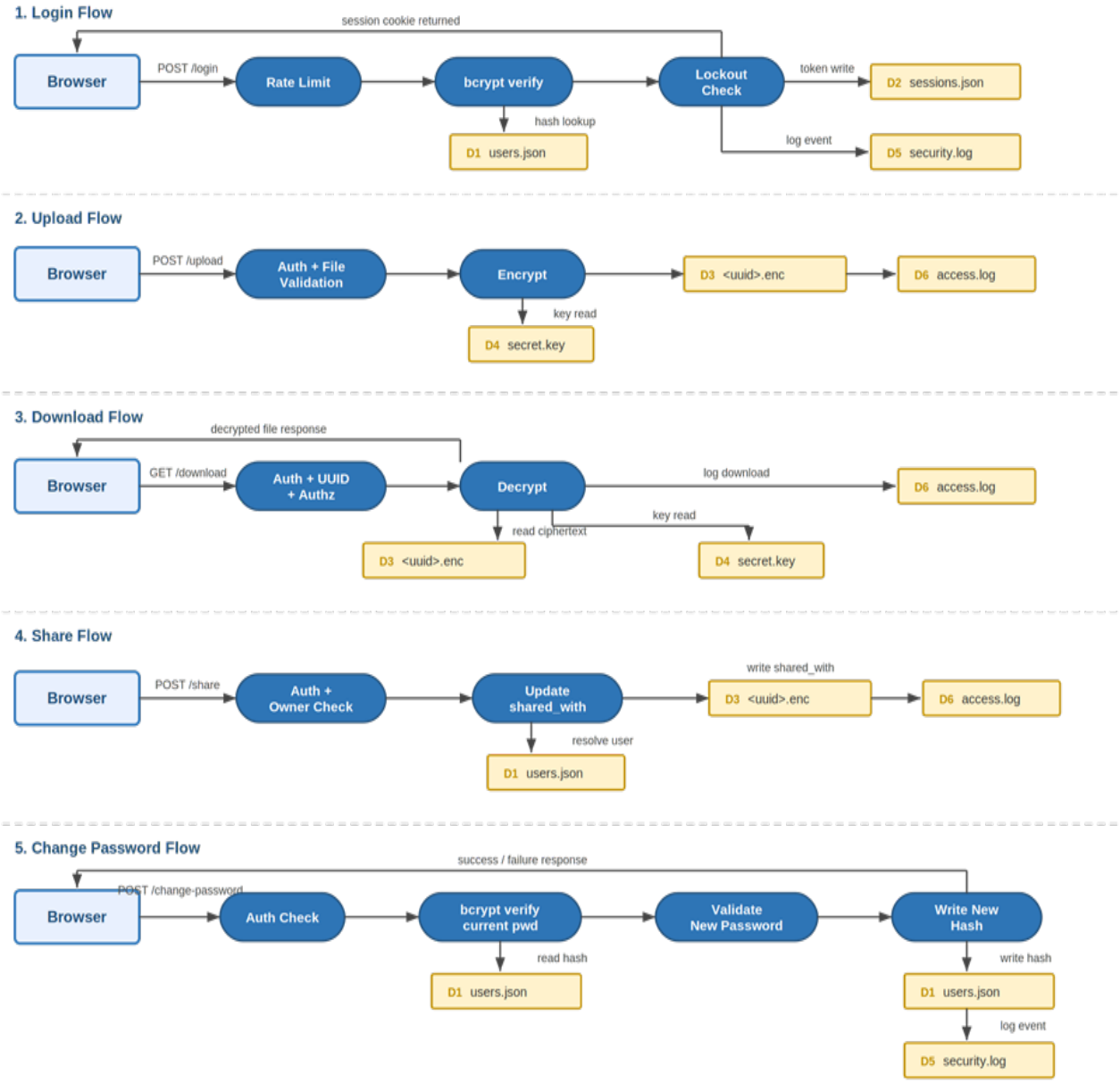
Figure A.2.1 — Context-Level Data Flow Diagram



A.2.2 Level-1 DFD

The Level-1 DFD decomposes the Flask Application process into the five security-sensitive flows: Login, Upload, Download, Share, and Change Password.

Figure A.2.2 — Level-1 Data Flow Diagram



A.2.3 Step-by-Step Flow Details

The five security-sensitive data flows are described step by step below.

Flow	Steps
Upload	1. Browser → POST /documents/upload (multipart, HTTPS) 2. Flask validates extension + MIME + size 3. File bytes base64-encoded then Fernet-encrypted 4. Ciphertext written to data/<uuid>.enc 5. Metadata

	(owner, timestamps, shared_with, versions) stored inside same .enc blob 6. Access log entry written
Download	1. Browser → GET /documents/download/<uuid> 2. Flask validates UUID format + path confinement 3. Session & document-level authorization checked (owner / editor / admin) 4. .enc file decrypted by EncryptedStorage 5. Raw bytes streamed as attachment; access logged
Login	1. Browser → POST /login 2. IP rate-limit check (10/min in-memory) 3. bcrypt.checkpw() against stored hash 4. On success: secrets.token_urlsafe(32) created, stored in sessions.json, token placed in signed Flask session cookie 5. On failure: failed_attempts incremented; lockout after 5 failures (15 min)
Share	1. Browser → POST /documents/share/<uuid> 2. Ownership verified (only owner can share) 3. Target username resolved to user_id 4. shared_with[user_id] = 'editor'|'viewer' written to .enc blob
Change Password	1. Browser → POST /change-password 2. Flask validates active session 3. Current password checked with bcrypt against users.json 4. New password validated against server-side policy 5. Password hash updated in users.json 6. PASSWORD_CHANGED or PASSWORD_CHANGE_FAILED logged to security.log 7. Response returned to browser

A.3 Component Descriptions

Component	Responsibility
app.py - Route layer	Handles all HTTP routing. Every route that touches user data is protected by @require_auth and, where applicable, @require_role.
EncryptedStorage	Wraps all document I/O. Every write calls Fernet.encrypt(); every read calls Fernet.decrypt(). Documents are never written to disk in plaintext.
StructuredLogger / SecurityLogger / AccessLogger	Two separate log streams: security.log for auth/access-control events; access.log for normal document operations. All entries are JSON-structured.
is_rate_limited(ip)	In-memory sliding-window counter. Allows a maximum of 10 login attempts per IP per 60-second window. Resets on server restart (acknowledged limitation).
require_auth decorator	Validates the Flask session cookie, looks up the token in sessions.json, and enforces a 30-minute inactivity timeout. Unauthenticated requests are redirected to /login.
require_role(role) decorator	Checks the role field on the current session. On mismatch, logs ACCESS_DENIED and returns HTTP 403.
securely_delete_file(path)	Overwrites a file with os.urandom(size) and calls fsync() before os.remove(). Best-effort on SSDs and journaling filesystems.
config.py	Centralises all tunable parameters: MAX_FAILED_ATTEMPTS, LOCKOUT_DURATION, RATE_LIMIT_MAX, SESSION_TIMEOUT,

	ALLOWED_EXTENSIONS, ALLOWED_MIME_TYPES, MAX_FILE_SIZE.

A.4 Technology Stack Justification

Choice	Justification
Python / Flask	Lightweight, well-documented framework. Jinja2 auto-escaping is on by default, reducing XSS risk. Large ecosystem of security libraries.
cryptography (Fernet)	Provides authenticated symmetric encryption (AES-128-CBC + HMAC-SHA256). Authentication means tampered ciphertext is detected at decryption time, not just at read time.
bcrypt (cost factor 12)	Deliberately slow KDF. Cost factor 12 makes offline brute-force attacks expensive.
JSON flat-file storage	Avoids an SQL injection surface entirely. No database parser to attack. Appropriate for project scale.
secrets.token_urlsafe(32)	Generates 256 bits of cryptographically random session tokens. Infeasible to guess or brute-force.
Flask signed session cookie	Used only to hold the server-side session token. The actual session data lives in sessions.json; the cookie is a pointer, not a payload.

B. Threat Model

B.1 Asset Identification

Asset	Sensitivity	Location
User credentials (password hashes)	High	data/users.json
Active session tokens	High	data/sessions.json
Uploaded document contents	High	data/<uuid>.enc (Fernet-encrypted)
User account metadata (email, role, lockout state)	Medium	data/users.json
Document sharing metadata	Medium	Inside each .enc blob
Document version history	Medium	Inside each .enc blob
Fernet encryption key	Critical	secret.key (local filesystem)
Security and access logs	Medium	logs/security.log, logs/access.log

B.2 Threat Enumeration

Threat	Category
Brute-force or credential-stuffing on /login	Authentication
Account enumeration via distinct registration error messages	Authentication / Information Disclosure
Session token theft via XSS or network sniffing	Session Hijacking
Replaying a session token after logout	Session Fixation / Replay
Normal user accessing another user's document by guessing doc_id (IDOR)	Horizontal Privilege Escalation
Normal user accessing /admin/dashboard or admin actions	Vertical Privilege Escalation
Path traversal via malformed doc_id to read arbitrary files	Input Validation / Path Traversal
Malicious file upload (executable, oversized, MIME spoofing)	File Upload Attack
XSS via stored or reflected user-controlled content	Cross-Site Scripting
CSRF - triggering state-changing actions from external sites	Cross-Site Request Forgery

Repeated guessing of current password on /change-password	Brute Force (authenticated)
Physical or forensic recovery of deleted document contents	Data Remanence
Theft of secret.key from compromised host	Key Compromise
DoS via resource exhaustion through rapid upload or login requests	Availability

B.3 Vulnerability Assessment

Threat	Likelihood	Impact
Brute-force on /login	Low (locked out after 5 attempts; IP rate-limited)	High if unchecked
Account enumeration via registration	Medium (distinct error messages exist in code)	Low-Medium
Session theft via XSS	Low (Jinja auto-escaping; HttpOnly cookie)	High if exploited
IDOR on document endpoints	Low (UUID validation + server-side ownership check)	High if exploited
Vertical privilege escalation	Low (@require_role checks on all admin routes)	High if exploited
Path traversal via doc_id	Low (UUID regex + abspath confinement)	High if exploited
Malicious file upload	Low (extension + MIME allowlist + 10 MB cap)	Medium
CSRF	Low (SameSite=Strict on session cookie)	Medium
No rate limit on /change-password	Medium (requires valid session first)	Low-Medium
Fernet key theft from disk	Conditional on host compromise	Critical

B.4 Attack Scenarios

Scenario 1 - Credential Stuffing

Attacker submits a list of known username/password pairs against /login. After 5 failures per account, the account locks for 15 minutes. After 10 attempts per minute from any one IP, further requests from that IP are rate-limited. The combination means a large-scale stuffing attack is

slowed to roughly 10 attempts per minute across any number of accounts, and each individual account locks after 5 failures.

Scenario 2 - IDOR on Document Download

Attacker authenticates as a normal user and crafts a request to `GET /documents/download/<uuid>` using a document UUID they guessed or found in logs. The route first validates the UUID format with a regex, then checks `file_data['user_id'] == user_id` and `shared_with.get(user_id) == 'editor'`. Neither condition is true, so the request returns 403 and a WARNING is logged.

Scenario 3 - Path Traversal via doc_id

Attacker submits `../../etc/passwd` as a doc_id. The route applies `re.match(r'^[a-f0-9\-]{36}$', doc_id)` - this does not match, so the request is immediately rejected with HTTP 400 and logged as PATH_TRAVERSAL_ATTEMPT.

Scenario 4 - Session Replay after Logout

Attacker captures a valid session cookie while authenticated, then waits for the user to log out. On logout, destroy_session(token) deletes the token from sessions.json. When the attacker replays the cookie, get_current_session() finds no matching token and returns None, so all protected routes redirect to login.

B.5 Risk Prioritization

Risk	Priority	Status
Fernet key stored on disk without extra protection	High	Accepted (out of scope for class project; documented)
Account enumeration via registration errors	Low	Open - see Known Limitations
No rate limiting on /change-password	Low	Open - see Known Limitations
unsafe-inline in CSP	Low	Open - see Known Limitations
In-memory rate limiter resets on restart	Low	Accepted for project scope
No dedicated CSRF token	Low	Mitigated by SameSite=Strict + session validation
Secure deletion not guaranteed on SSDs	Informational	Mitigated by Fernet encryption as primary defense

C. Security Controls

C.1 User Authentication (15 pts)

C.1.1 Password Hashing

Passwords are hashed with `bcrypt.hashpw(..., bcrypt.gensalt(rounds=12))` before storage. The plaintext password is never written to disk or logged. Cost factor 12 makes offline brute-force expensive.

C.1.2 Password Complexity

validate_password() enforces: minimum 12 characters, at least one uppercase letter, one lowercase letter, one digit, and one special character from !@#$%^&*. This check runs server-side on both registration and password change.

C.1.3 Account Lockout & Rate Limiting

- After 5 consecutive failed logins, locked_until is set to now + LOCKOUT_DURATION (15 minutes). Locked accounts cannot log in until the timer expires or an admin unlocks them.
- is_rate_limited(ip) maintains an in-memory sliding window: if an IP sends ≥ 10 login POSTs in 60 seconds, it is blocked for the remainder of that window.

C.1.4 Session Creation

Successful login calls `secrets.token_urlsafe(32)` (256 bits of entropy). The token is stored in sessions.json server-side and placed in a Flask signed session cookie configured with HttpOnly=True, Secure=True, SameSite='Strict'.

> Testing: Submitted 6 consecutive wrong passwords → account locked. Submitted 11 rapid login attempts from same IP → rate-limit message returned. Captured cookie, logged out, replayed cookie → redirected to login.

C.2 Access Control (15 pts)

C.2.1 Role-Based Access Control

Three user roles are enforced (admin, user, guest), plus document-level permissions (owner, editor, viewer). The user roles and their base permissions are shown below.

Action	Admin	User	Guest
Register / log in	✓	✓	✓
Create own content	✓	✓	✗
View own content	✓	✓	✗

Edit own content	✓	✓	✗
Delete own content	✓	✓	✗
Share own content	✓	✓	✗
View shared content	✓	✓	✓*
Edit shared content	✓	✓	✗
View all content	✓	✗	✗
Manage users	✓	✗	✗

** Guests can view documents that have been explicitly shared with them (as viewer), but cannot create, edit, delete, or share content. Owner, editor, and viewer are document-level access permissions, not account roles.*

C.2.2 Document-Level Authorization

Route-level RBAC is a necessary but not sufficient guard. Every document route also performs an object-level check: is the requesting user the owner, an admin, or listed in shared_with with the appropriate role? This prevents horizontal privilege escalation - a normal user cannot access another normal user's document even with a valid session.

Sharing is further restricted: only the document owner may call `POST /documents/share/<uuid>`. If the requesting user is not the owner, the request is denied and logged.

> Testing: Logged in as user B and attempted GET /documents/download/<user A's doc UUID> → 403 returned, ACCESS_DENIED logged. Requested /admin/dashboard as normal user → 403 returned.

C.3 Input Validation & Injection Prevention (20 pts)

C.3.1 XSS Prevention

Jinja2 auto-escaping is enabled by default for all `{{ }}` template expressions. Usernames are further restricted to `r'^\w{3,20}$'` (letters, digits, underscores only), eliminating the most common XSS payloads from that field. The `| safe` filter is not used anywhere in the templates.

C.3.2 Path Traversal Prevention

All document routes validate the doc_id parameter with `re.match(r'^[a-f0-9\-]{36}$', doc_id)` before constructing the filesystem path. After construction, `os.path.abspath(enc_path).startswith(os.path.abspath('data'))` is checked as a second layer. If either check fails, HTTP 400 is returned and PATH_TRAVERSAL_ATTEMPT is logged.

C.3.3 Command Injection Prevention

No subprocess, os.system, popen, eval, or exec calls exist in the codebase. User input is never passed to shell commands. All file operations use Python's standard library with sanitized, validated paths.

C.3.4 File Upload Validation

allowed_file() checks both the file extension (`ALLOWED_EXTENSIONS = {pdf, txt, docx, png, jpg, jpeg}`) and the MIME type (`ALLOWED_MIME_TYPES`) reported by the browser. Files larger than `MAX_FILE_SIZE = 10 MB` are rejected before the content is processed. Rejected uploads are logged as UPLOAD_REJECTED.

C.3.5 Input Length and Format Limits

- Username: 3-20 characters, alphanumeric + underscore only (regex enforced server-side)
- Email: basic format validation via regex (presence of @ and domain dot)
- Password: minimum 12 characters with complexity requirements
- Sharing role: validated against a whitelist - only 'editor' or 'viewer' accepted

> Testing: Submitted <script>alert(1)</script> as a username → rejected by regex. Attempted ../../etc/passwd as doc_id → HTTP 400, logged. Uploaded .exe file → rejected. Uploaded valid file over 10 MB → rejected.

C.4 Encryption (15 pts)

C.4.1 Data-at-Rest Encryption

The `EncryptedStorage` class wraps all document I/O. On write, `Fernet.encrypt()` produces an authenticated ciphertext (AES-128-CBC + HMAC-SHA256). The ciphertext is base64-encoded and stored as the `data` field inside each `.enc` file. On read, `Fernet.decrypt()` verifies the HMAC before returning the plaintext - tampered ciphertext raises an exception rather than returning garbage data.

The Fernet key is generated with `Fernet.generate_key()` on first run and persisted to `secret.key`. This file is loaded at application startup and never regenerated unless deleted.

C.4.2 Transport Encryption

The application runs with `ssl_context=('cert.pem', 'key.pem')`. The `require_https()` before_request hook redirects any HTTP request to HTTPS with a 301 redirect, unless `app.debug` or `app.testing` is True.

> Testing: Opened a .enc file in a hex editor - confirmed only ciphertext visible, no plaintext filename or content. Modified one byte of the ciphertext, restarted app, and attempted to load the file - Fernet raised InvalidToken exception, the file was logged as FILE_LOAD_ERROR.

C.5 Session Management (15 pts)

C.5.1 Session Creation and Storage

On successful login, `create_session(user)` generates a 256-bit token via `secrets.token_urlsafe(32)`. The token is stored server-side in `data/sessions.json` alongside the user_id, username, role, created_at, and last_activity timestamps. The token is placed in a signed Flask session cookie.

C.5.2 Session Validation and Timeout

Every protected route calls `get_current_session()`, which reads the token from the Flask session cookie, looks it up in sessions.json, and checks `time.time() - last_activity > SESSION_TIMEOUT (1800 s)`. An expired session is deleted server-side and the client cookie is cleared. last_activity is updated on every valid request, implementing a sliding window.

C.5.3 Logout and Session Invalidation

Logout calls `destroy_session(token)`, which removes the token from sessions.json, then calls `session.clear()` to remove the client-side cookie. Replaying the old token after logout finds no matching entry in sessions.json and returns None.

C.5.4 Cookie Security Flags

- HttpOnly=True - cookie is inaccessible to JavaScript, reducing XSS-based session theft
- Secure=True - cookie is only sent over HTTPS connections
- SameSite='Strict' - cookie is not sent on cross-site requests, providing CSRF protection without a dedicated token

> Testing: Logged in, captured session token, logged out, replayed token in a fresh request → redirected to login. Confirmed session entry was absent from sessions.json after logout. Verified cookie flags in browser DevTools.

C.6 Security Headers (10 pts)

All responses include the following headers, set in the set_security_headers() after_request hook:

Header	Value and Purpose
Content-Security-Policy	default-src 'self'; script-src 'self' 'unsafe-inline'; style-src 'self' 'unsafe-inline'; img-src 'self' data:; font-src 'self'; connect-src 'self'; frame-ancestors 'none' Note: unsafe-inline is a known limitation (see Section C.8)
X-Frame-Options	DENY - prevents clickjacking via iframe embedding

X-Content-Type-Options	nosniff - prevents MIME-type sniffing attacks
X-XSS-Protection	1; mode=block - legacy XSS filter for older browsers
Referrer-Policy	strict-origin-when-cross-origin - limits referrer leakage
Permissions-Policy	geolocation=(), microphone=(), camera=() - disables unused browser APIs
Strict-Transport-Security	max-age=31536000; includeSubDomains - enforces HTTPS for 1 year

C.7 Logging & Monitoring (10 pts)

Two separate loggers write JSON-structured entries to distinct log files:

Logger	Events Captured
SecurityLogger → logs/security.log	LOGIN_FAILED, LOGIN_BLOCKED, ACCOUNT_LOCKED, SESSION_CREATED, SESSION_DESTROYED, ACCESS_DENIED, UPLOAD_REJECTED, PATH_TRAVERSAL_ATTEMPT, PASSWORD_CHANGE_FAILED, PASSWORD_CHANGED, ADMIN_USER_LOCKED, ADMIN_USER_UNLOCKED, RATE_LIMITED, FILE_LOAD_ERROR
AccessLogger → logs/access.log	LOGIN_SUCCESS, LOGOUT, DASHBOARD_ACCESSED, DOCUMENTS_ACCESSED, ADMIN_DASHBOARD_ACCESS, FILE_UPLOADED, FILE_VERSION_UPLOADED, FILE_DOWNLOADED, FILE_VERSION_DOWNLOADED, FILE_VIEWED, FILE_VERSION_RESTORED, DOCUMENT_SHARED, DOCUMENT_DELETED, DOCUMENT_VERSIONS_ACCESSED

Each log entry is a JSON object containing: timestamp (UTC ISO-8601), event_type, user_id, ip_address, and a details dict. Severity levels map to Python logging levels: INFO, WARNING, ERROR, CRITICAL.

C.8 Known Limitations and Mitigations

Limitation	Risk Level	Mitigation / Notes
Fernet key in secret.key (no KMS)	High if host is compromised	Encryption still protects data at rest from disk theft. Move key to environment variable or secrets manager for any production deployment.
Account enumeration via registration (distinct error messages for username vs. email)	Low	Collapse to a single generic message: 'An account with that username or email already exists.'

No rate limiting on /change-password	Low (requires valid session)	Apply the same is_rate_limited() pattern already used on /login.
CSP allows unsafe-inline for scripts and styles	Low	Move all inline JS/CSS to external files and tighten the CSP. This is the standard path to a stricter policy.
In-memory rate limiter resets on server restart	Low	Move to a persistent store (Redis or a simple JSON counter) for production.
No dedicated CSRF token on forms	Low	SameSite=Strict mitigates most CSRF risk. A library such as Flask-WTF would add explicit token protection.
Secure deletion is best-effort on SSDs / journaling filesystems	Informational	Fernet encryption is the primary defense. Physical erasure guarantees are outside application scope.
Multiple concurrent sessions allowed per user	Informational	Acceptable for this project. Could add single-session enforcement by deleting old tokens on new login.

D. Data Protection

D.1 Data Classification

Data Type	Classification	Justification
Document file contents	Confidential	User-uploaded private documents. Highest protection required.
Password hashes	Confidential	Compromise enables offline cracking. Must never be transmitted or logged.
Active session tokens	Confidential	Equivalent to a password for the duration of the session.
Fernet encryption key	Secret / Critical	Compromise decrypts all stored documents. Requires strongest protection.
User metadata (email, role, lockout)	Sensitive	PII and account control data. Protected at rest in users.json.
Document sharing metadata	Sensitive	Reveals who has access to what. Stored inside the encrypted .enc blob.
Document version history	Sensitive	Previous document content. Also encrypted inside the .enc blob.
Security event logs	Internal	Contains IPs and usernames. Should not be publicly accessible.
Access logs	Internal	Normal operation records. Useful for audit; not secret but not public.

D.2 Encryption Methods

What	Algorithm	Implementation Detail
Document contents (at rest)	Fernet (AES-128-CBC + HMAC-SHA256)	EncryptedStorage.save_encrypted() - every write is a fresh encryption with a new IV. Authenticated encryption detects tampering.
Password storage	bcrypt (cost factor 12)	hash_password() - salted, slow KDF. Verification via bcrypt.checkpw() only; hash never logged.
Session token generation	CSPRNG - secrets.token_urlsafe(32)	256 bits of entropy. Stored server-side; only token pointer sent to client.
Transport (in transit)	TLS via ssl_context=('cert.pem','key.pem')	Self-signed certificate for dev. Production deployment requires a CA-signed certificate.

D.3 Key Management

The Fernet key lifecycle is as follows:

1. Generation: On first startup, Fernet.generate_key() produces a 32-byte URL-safe base64 key backed by os.urandom(). It is written to secret.key.
2. Storage: The key is stored in secret.key in the project root. Access is controlled by OS filesystem permissions. The key is not committed to version control (secret.key is in .gitignore).
3. Loading: At application startup, EncryptedStorage reads the key once and holds it in memory as self.key. The key is not re-read on every operation.
4. Rotation: Manual only. Rotating the key requires re-encrypting all .enc files with the new key. No automated rotation is implemented for this project.
5. Backup: None for the class project. In production, the key must be backed up securely - losing it makes all .enc files permanently unreadable.

> Production recommendation: Replace the file-based key with a secret injected at runtime via an environment variable (FERNET_KEY) or a dedicated secrets manager. This removes the key from the filesystem entirely.

D.4 Secure Deletion Procedures

When a document is deleted via `POST /documents/delete/<uuid>`, the application calls `securely_delete_file(path)`, which:

6. Reads the current file size with os.path.getsize()
7. Opens the file in read+write binary mode
8. Overwrites the entire file with os.urandom(size)
9. Calls f.flush() and os.fsync(f.fileno()) to push the overwrite to the OS
10. Calls os.remove() to remove the directory entry

This approach provides reasonable protection on traditional magnetic hard drives. On SSDs and journaling filesystems, wear leveling and the filesystem journal may retain data in blocks outside the overwritten region. The primary defense against data exposure is therefore Fernet encryption: even if the raw bytes are recovered, they are ciphertext without the key.

Version history blobs are stored inside the same .enc file as the current document version. Deleting the document deletes all versions simultaneously.

> For production deployments where stronger erasure guarantees are required, full-disk encryption or a hardware-backed key store is the recommended approach. Application-layer overwrite alone is insufficient on modern storage hardware.